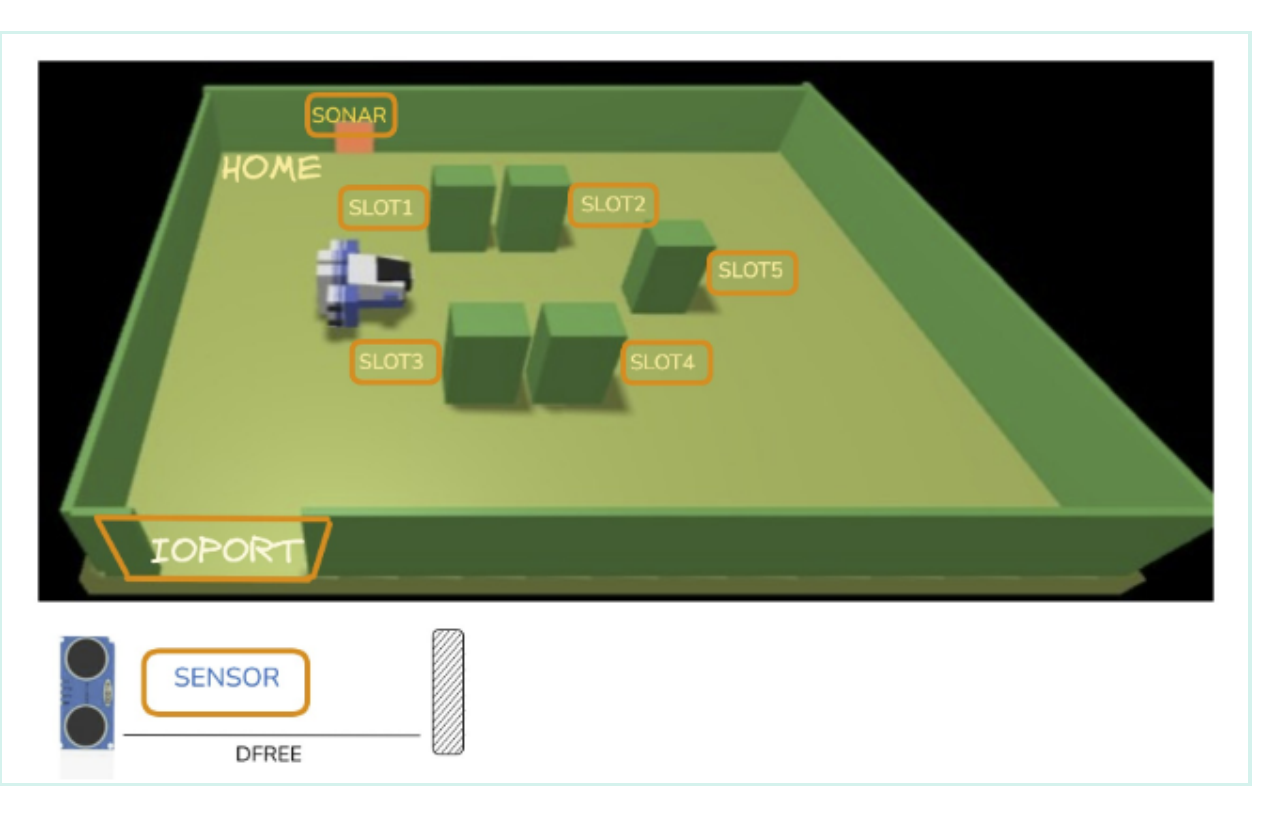
TF26 - Sprint1

Goal Sprint1: Sviluppare un prototipo software funzionante incentrato sul componente cargoservice. Il sistema dovrà essere in grado di elaborare stimoli di input fittizi (che simulano la pressione del pulsante della IOPort, *button_pushed*), aggiornare coerentemente lo stato di occupazione degli slot della stiva (*hold_status*) e governare i cicli di temporizzazione per il disimpegno del sistema.

Introduction

A *Maritime Cargo shipping company* (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply **hold**). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**).

The *hold* is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides `4 slots` to store the containers and a slot named **slot5**.



In the picture above:

- The **slots1-4** depict the hold areas reserved to store one *container* each,
- The **slot5** depicts an area, where the *cargorobot* must temporarily store a container, before to place it in one of the *slots1-4*. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.
- The **IOPort** is a device with a **pushbutton** and **a display**. The *pushbutton* is pressed by the customer in order to to send a request to load a container on the cargo. The *display* is used to show the answer to the request and to show the current state of the *hold*.
- The **sensor** associated to the *IOPort* is a device (a *sonar*) used to detect the presence of a container, when it measures a distance D, such that `D < DFREE/2`, during a reasonable time (e.g. 3 secs).

Requirements

The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer `retrylater`, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the `slots1-4` are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request* to load is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message **'Service working'**, when it is all is going well
- the message **'Out of service'** if the *sonar sensor* measures a distance `D > DFREE` for at least 3 secs (perhaps a failure of the *sonar*).

Requirement analysis

L'analisi dei requisiti è stata svolta nello Sprint0.

Problem analysis

In questo Sprint, focalizzato sul core del servizio e sulla gestione dell'interazione con l'IOPort, l'analisi del problema affronta le criticità architetturali e tecnologiche derivanti dal passaggio a un sistema distribuito ed eterogeneo.

Individuazione dei componenti del sottosistema

In questa fase analizziamo la natura computazionale dei componenti coinvolti nel perimetro dello Sprint 1:

- **cargoservice**: come evidenziato nello Sprint 0, è il cuore della logica di business e governa una macchina a stati complessa. Non può essere un componente passivo (POJO) perché deve essere reattivo a messaggi asincroni (la richiesta dell'utente) e al contempo proattivo nella gestione dei vincoli temporali (deve svegliarsi autonomamente allo scadere del timeout di 30 secondi senza sollecitazioni esterne). Viene quindi modellato come QActor.
- **ioport**: è l'entità di frontiera che interfaccia il mondo esterno (utente/browser) con il core applicativo. Gestisce stimoli concorrenti bidirezionali: cattura l'azione dell'utente (click sul pulsante) per inoltrare la richiesta a cargoservice, e riceve gli aggiornamenti di stato per notificarli all'interfaccia. Anch'esso deve essere modellato come QActor.
- **hold**: rappresenta lo stato degli alloggiamenti della stiva. Dato che non possiede un thread di controllo autonomo né reagisce spontaneamente a stimoli esterni, ma viene interrogata e manipolata in modo sequenziale unicamente dall'attore cargoservice, è opportunamente modellata come un POJO Java puro.

Scelta della tecnologia per l'IOPort (Web GUI vs dispositivo dedicato)

Come già discusso nei requisiti, l'IOPort deve presentare un'interfaccia per il cliente finale con un display informativo e un pulsante di richiesta. Valutando le alternative:

1. **Terminale da riga di comando / Mock testuale**: semplice da realizzare ma inadatto a rappresentare l'interazione reale richiesta dal committente (utente finale).
2. **Pannello embedded dedicato**: introduce forti vincoli hardware e vincola l'accesso a una singola postazione fisica.
3. **WebGUI accessibile via Browser**: opzione preferenziale. Permette di simulare realisticamente sia l'interfaccia cliente sia eventuali controlli di test (es. simulatore dello stato del sensore), riduce i costi infrastrutturali e consente una visualizzazione contemporanea e distribuita dello stato del sistema.

Il divario tecnologico tra Browser e QAk: perché non basta TCP nativo e perché si esclude il Polling

L'adozione di un'interfaccia Web introduce un disallineamento (abstraction & technology gap) rispetto all'ecosistema QAk:

- **Limitazioni del Browser**: il runtime QAk comunica tramite socket TCP scambiandosi messaggi formattati secondo la sintassi proprietaria IApplMessage. I browser web, per motivi di sicurezza legati alle policy di sandboxing, non consentono al codice JavaScript di aprire socket TCP/IP grezzi.
- **Scarto del Polling HTTP**: un approccio basato su richieste periodiche (HTTP GET ogni N secondi) introdurrebbe latenza non deterministica nella notifica del cambio di stato (es. notifica immediata di timeout o accettazione richiesta) e genererebbe un traffico di rete inutile e scalabile negativamente all'aumentare dei client connessi.
- **Scelta di WebSocket**: si adotta il protocollo WebSocket. Esso fornisce un canale full-duplex su una singola connessione TCP persistente, permettendo al server di inviare aggiornamenti push asincroni in tempo reale alla GUI non appena lo stato cambia, senza richiedere componenti middleware esterni (come un broker MQTT) che appesantirebbero inutilmente l'architettura di questo sprint.

Il componente Ponte (IOPortWsAdapter) e la concorrenza tra Thread

La gestione del socket WebSocket introduce una criticità architetturale legata al modello a memoria isolata degli attori:

La libreria del server web/WebSocket (es. Javalin) elabora le connessioni e i messaggi in arrivo su un proprio pool di thread worker, distinto dal thread su cui gira il ciclo di vita (loop di ricezione) dell'attore ioport.

Nel modello QAk, nessun thread esterno può invocare direttamente metodi interni di un attore per mutarne lo stato, pena la violazione delle garanzie di thread-safety e la corruzione della mailbox.

Soluzione: si introduce una classe ausiliaria Java/Kotlin, IOPortWsAdapter, di proprietà esclusiva di ioport. L'adapter riceve gli eventi dal thread WebSocket e, per consegnarli all'attore, utilizza l'unico canale lecito e thread-safe previsto dal runtime: l'invocazione di `sendMsgToMyself(dispatch)`.

Il sistema distribuito e le problematiche di comunicazione

Con l'introduzione di entità web e di nodi separati, il sistema evolve in un'architettura distribuita: è quindi necessaria la rete per permettere la comunicazione tra le varie parti. Questo comporta un cambio di paradigma rispetto ai sistemi monolitici basati su chiamate procedurali all'interno di una singola JVM.

Formato dei messaggi e flussi di comunicazione

La gestione dei messaggi riflette la separazione tra il mondo eterogeneo del browser (basato su stringhe e canali WebSocket in ossequio ai principi della Clean Architecture) e il mondo formale degli attori QAk. L'attore ioport svolge il ruolo fondamentale di mediatore e traduttore di frontiera, evitando che la logica di business centrale sia accoppiata ai dettagli tecnologici della GUI.
I messaggi si suddividono in base alla natura dell'interazione:

1. **Messaggi tra l'attore ioport e il cargoservice**
   Quando l'attore ioport riceve il dispatch button_pushed dalla GUI, non si limita a duplicare l'informazione, ma la traduce in una richiesta formale di servizio (Request), poiché l'IOPort si aspetta una risposta vincolante dal sistema centrale, come richiesto dai requisiti. I messaggi necessari erano già stati delineati nell'analisi dei requisiti:
   - `loadrequest : loadrequest(IOPORT_STATE)` (Request): Inviata da ioport a cargoservice per richiedere l'inserimento di un container, passando lo stato corrente della porta.
   - `engaged : engaged(RESERVED_SLOT, HOLD_STATE)` (Reply): Risposta di accettazione con cui il cargoservice assegna uno slot libero e restituisce lo stato aggiornato della stiva.
   - `reject : reject(HOLD_STATE)` (Reply): Risposta di rifiuto emessa quando la stiva è satura (slot 1-4 occupati).
   - `retrylater : retrylater(CAUSE, HOLD_STATE)` (Reply): Risposta di attesa inviata se la porta è momentaneamente occupata o il sistema si trova in condizioni particolari.

   Per la gestione asincrona degli aggiornamenti interni (come i timeout o le notifiche autonome di display), il cargoservice utilizza specifici Dispatch di servizio verso l'ioport:
   - `updateHoldDisplay : updateHoldDisplay(STATE, HOLD_STATE, MSG)`
   - `updateWorkingState : updateWorkingState(STATE)`

   Questi messaggi sono utilizzati in modo asincrono per consentire al cargoservice di notificare autonomamente all'ioport variazioni di stato o scadenze temporali (come il timeout dei 30 secondi), senza che vi sia una richiesta esplicita contestuale da parte dell'utente.
2. **Messaggi tra Web GUI e l'attore ioport**
   Mentre tra gli attori ioport e cargoservice si impiega la semantica nativa QAk (Request/Reply e Dispatch), sul canale WebSocket verso il browser si sceglie il formato String di IApplMessage:

   ```
   msg(button_pushed, dispatch, webgui, ioport, X, N)
   msg(hold_status, dispatch, ioport, webgui, SLOT_STATES, N)
   msg(working_state, dispatch, ioport, webgui, STATE, N)
   ```

   In conformità con il principio della Clean Architecture, si è preferito il formato String piuttosto che quello JSON, in quanto in questo modo è la pagina Web ad adattarsi all'applicazione e non viceversa.
   L'adapter si occupa della serializzazione e deserializzazione, mantenendo il frontend completamente agnostico rispetto alla sintassi IApplMessage di QAk.
   La Web GUI non invia comandi diretti al servizio centrale, ma genera eventi asincroni di frontiera:
   - `button_pushed`: Dispatch inviato dalla GUI (tramite l'adapter) all'attore ioport per notificare la pressione del pulsante da parte dell'utente.
   - `hold_status` / `working_state`: Dispatch di ritorno con cui l'attore ioport aggiorna i componenti visivi della pagina web in base agli stati elaborati dal sistema.

Modello dall'analista del problema

A conclusione dell'analisi del problema, al fine di formalizzare il comportamento dinamico del sistema e le interazioni logiche tra gli attori senza dipendere da dettagli tecnologici prematuri, viene definito il Modello dell'Analista del Problema.
Questo modello estende i requisiti introducendo la struttura formale e le macchine a stati degli attori cargoservice e ioport, la gestione dei vincoli temporali (come il timeout di 30 secondi) e lo scambio dei messaggi logici essenziali. Esso costituisce il ponte concettuale ideale tra la specifica funzionale di alto livello e la successiva fase di progettazione e implementazione tecnologica.

```
System cargoservice

/*
 * MODELLO DELL'ANALISTA DEL PROBLEMA
 */

// -----definizione dei messaggi-----

// tra l'attore ioport e il cargoservice
Request loadrequest     : loadrequest(IOPORT_STATE)
Reply retrylater        : retrylater(CAUSE, HOLD_STATE) for loadrequest
Reply reject            : reject(HOLD_STATE) for loadrequest
Reply engaged           : engaged(RESERVED_SLOT, HOLD_STATE) for loadrequest
Dispatch updateHoldDisplay : updateHoldDisplay(STATE, HOLD_STATE, MSG)
Dispatch updateWorkingState : updateWorkingState(STATE)

// tra Web GUI e l'attore ioport
Dispatch button_pushed : button_pushed(X)
Dispatch hold_status : hold_status(SLOT_STATES)
Dispatch working_state : working_state(STATE)

// -----definizione del contesto-----
Context ctxcargoservice ip [host="localhost" port=8120]

// -----definizione degli attori-----
QActor cargoservice context ctxcargoservice{
	[#
		var HOLD_FULL = false
		var ENGAGED = false
		var WORKING = true
	#]
	State s0 initial{
		println("$name | starts ") color magenta
	}
	Goto idle

	State idle{
		[# var Msg = "cargoservice partito"#]
		println("$name | Sistema in stato IDLE") color magenta
		forward ioport -m updateHoldDisplay : updateHoldDisplay($ENGAGED, $HOLD_FULL, $Msg)
		forward ioport -m updateWorkingState : updateWorkingState($WORKING)
	}
	Transition t0
		whenRequest loadrequest -> handleLoadRequest

	State handleLoadRequest{
		onMsg(loadrequest : loadrequest(IOPORT_STATE)) {
	  [#
		var IoportState = payloadArg(0).toBoolean()

		if (IoportState) {
			var Cause = "porta_occupata"
		  #]
		  replyTo loadrequest with retrylater : retrylater($Cause, $HOLD_FULL)
		  println("$name | retrylater($Cause, $HOLD_FULL)") color magenta
		  [#
		} else if (HOLD_FULL) {
		  #]
		  replyTo loadrequest with reject : reject($HOLD_FULL)
		  println("$name | reject($HOLD_FULL)") color magenta
		  [#
		} else {
		  HOLD_FULL = true
		  var Slot = "slot1"
		  ENGAGED = true
		  #]
		  replyTo loadrequest with engaged : engaged($Slot, $HOLD_FULL)
		  println("$name | engaged($Slot, $HOLD_FULL)") color magenta
		  [#
		}
	  #]
	}
	}
	Goto engagedState if [# ENGAGED #] else idle

	State engagedState{
		println("$name | Sistema in stato IDLE") color magenta
	}
	Transition t0
		whenTime 30000 -> disengagedState
		whenRequest loadrequest -> handleBusyRequest
		// whenMsg occupazione del sensore -> gestione del carico

	State handleBusyRequest {
		[#
			var Cause = "sistema_engaged"
	#]
	replyTo loadrequest with retrylater : retrylater($Cause, $HOLD_FULL)
	println("$name | retrylater($Cause, $HOLD_FULL)") color magenta
  }
  Goto engagedState

	State disengagedState{
		println("$name | Sistema in stato DISENGAGED") color magenta
		[#
			ENGAGED = false
			var Msg = "sistema_disengaged"
		#]
		forward ioport -m updateHoldDisplay : updateHoldDisplay($ENGAGED, $HOLD_FULL, $Msg)
	}
	Goto idle
}

QActor ioport context ctxcargoservice{
	[#
		var IoportState = false
	#]
	State s0 initial{
		println("$name | starts ") color blue
	}
	Goto acceptEvents

	State acceptEvents{
		println("$name | in attesa di eventi...") color blue
	}
	Transition t0
	whenMsg button_pushed       -> handleButton
	whenMsg updateHoldDisplay   -> handleCoreHoldUpdate
	whenMsg updateWorkingState  -> handleCoreWorkingState

	State handleButton{
		println("$name | invio loadrequest a cargoservice, ioport libera") color blue
		request cargoservice -m loadrequest : loadrequest($IoportState)
	}
	Transition t1
	whenReply engaged    -> handleEngaged
	whenReply reject     -> handleReject
	whenReply retrylater -> handleRetryLater

    State handleEngaged{
	printCurrentMessage color blue
	// comunicazione di Hold Status tramite Adapter
    }
    Goto acceptEvents

    State handleReject{
	printCurrentMessage color blue
	// comunicazione di Hold Status tramite Adapter
    }
    Goto acceptEvents

    State handleRetryLater{
	printCurrentMessage color blue
	// comunicazione di Hold Status tramite Adapter
    }
    Goto acceptEvents

    State handleCoreHoldUpdate{
	printCurrentMessage color blue
	// comunicazione di Hold Status tramite Adapter
    }
    Goto acceptEvents

    State handleCoreWorkingState{
	printCurrentMessage color blue
	// comunicazione di Working State tramite Adapter
    }
    Goto acceptEvents
}
```

Test plans

I test plan per lo Sprint 1 hanno lo scopo di verificare in modo riproducibile e formale che la sequenza di stimoli produca esattamente le transizioni e le emissioni di messaggi previste dai requisiti.

Caso di test	Precondizione	Input/Azione	Risultato atteso
TC1: Richiesta accettata	Almeno 1 slot `FREE` nella Hold; sensore non occupato	Ricezione dispatch `button_pushed` da `ioport`	`cargoservice` invia reply `engaged(RESERVED_SLOT, STATE)`; passa da idle a engaged; `ioport` riceve e notifica la GUI
TC2: Rifiuto IOPort occupato	Flag sensore su occupato (`setOccupied(true)`)	Ricezione dispatch `button_pushed` da `ioport`	`cargoservice` invia reply `retrylater(CAUSE, HOLD_STATE)`; lo stato resta invariato (idle)
TC3: Rifiuto stiva piena	Tutti e 4 gli slot occupati/riservati nella Hold	Ricezione dispatch `button_pushed` con sensore libero	`cargoservice` risponde con `reject(HOLD_STATE)`; nessuno slot viene modificato
TC4: Timeout 30 secondi	Sistema appena passato a engaged, slot riservato	Nessun evento di carico per 30000 ms (`whenTime 30000`)	`cargoservice` invoca `releaseSlot()` sulla Hold, emette dispatch `updateHoldDisplay` verso ioport con messaggio di timeout e torna allo stato idle

Project

Nello spirito del corso, l'architettura logica e il comportamento dinamico del prototipo sono formalizzati primariamente nel modello eseguibile `sprint1.qak`, da cui il compilatore/generatore del runtime QAk produce il codice sorgente Kotlin degli attori.
I componenti accessori a supporto (la struttura dati Hold, l'adapter di rete IOPortWsAdapter e le risorse web della GUI) sono confinati in classi di utilità Java/Kotlin e file statici ben isolati.

L'utilizzo di componenti e messaggi mock in questa fase

Per consentire lo sviluppo e il test rigoroso del cargoservice e della Web GUI in questo Sprint 1 — in attesa di integrare i dispositivi hardware reali (sonar, LED e robot fisico) previsti nei successivi sprint — sono stati introdotti specifici messaggi e meccanismi di Mock:

- Simulazione del sensore (`containerSensed`): Attraverso un apposito comando o interruttore di controllo presente sulla Web GUI, è possibile simulare la variazione di distanza rilevata dal sonar (DFREE), iniettando l'evento di presenza container che attiva la presa in carico del sistema.
- Simulazione di guasto (`outOfService`): Permette di testare la reattività del core di fronte a condizioni anomale o malfunzionamenti del sensore, forzando la transizione allo stato protetto di blocco.
- Gestione dello stato visivo (`led`): Sincronizza lo stato dell'indicatore luminoso (spento, lampeggiante, rosso di errore) tra il core, l'attore di frontiera e la rappresentazione grafica sul browser.

L'attore cargoservice e la gestione della stiva (Hold)

L'attore gestisce la logica di business e governa il timeout di 30 secondi per la transizione tra gli stati idle, engaged e disengaged. Incapsula l'oggetto Hold mediante la direttiva withobj.

La classe Java Hold agisce come POJO strutturato e offre metodi sicuri e sincroni invocati dal core per verificare se la stiva è piena (isFull()), prenotare lo slot libero successivo (reserveNextFreeSlot()), marcare lo slot come occupato o rilasciarlo in caso di timeout o errore.

```
QActor cargoservice context ctxcargoservice
    withobj h using "cargoservice.Hold()" {
    [#
        var CurrentReservedSlot = -1
        var CurrentState = "IDLE"
        var IsOutOfService = false
        var WaitStartTime = 0L
        var RemainingTime = 30000L
    #]

    State s0 initial {
        println("$name | avvio CargoService Core") color magenta
        [# var ServiceWorking = "'WORKING'" #]
        forward ioport -m updateWorkingState : updateWorkingState($ServiceWorking)
    }
    Goto idle

    State idle {
        [#
            CurrentState = "IDLE"
            var LedOff = "'off'"
            WaitStartTime = 0L
            RemainingTime = 30000L
        #]
        println("$name | Sistema in stato IDLE") color magenta
        forward ioport -m led : led($LedOff)
    }
    Transition t0
        whenRequest loadrequest -> handleLoadRequest
        whenMsg outOfService -> handleOutOfService

    State handleLoadRequest {
        onMsg(loadrequest : loadrequest(IOPORT_STATE)) {
            [#
                var IoportState = payloadArg(0).toBoolean()
                var CurrentHoldState = h.getHoldStateString()

                if (IoportState) {
                    var SystemBusyStr = "'ioport_occupied'"
                    #]
                    replyTo loadrequest with retrylater : retrylater($SystemBusyStr, $CurrentHoldState)
                    println("$name | retrylater($SystemBusyStr, $CurrentHoldState)") color magenta
                    [#
                } else if (h.isFull()) {
                    #]
                    replyTo loadrequest with reject : reject($CurrentHoldState)
                    println("$name | reject($CurrentHoldState)") color magenta
                    [#
                } else {
                    CurrentReservedSlot = h.reserveNextFreeSlot()
                    if (CurrentReservedSlot != -1) {
                        CurrentState = "ENGAGED"
                        CurrentHoldState = h.getHoldStateString()
                        var LedBlinking = "'blinking'"
                        #]
                        replyTo loadrequest with engaged : engaged($CurrentReservedSlot, $CurrentHoldState)
                        println("$name | engaged($CurrentReservedSlot, $CurrentHoldState)") color magenta
                        forward ioport -m led : led($LedBlinking)
                        [#
                    } else {
                        var SystemBusyStr = "'system_busy'"
                        #]
                        replyTo loadrequest with retrylater : retrylater($SystemBusyStr, $CurrentHoldState)
                        println("$name | retrylater($SystemBusyStr, $CurrentHoldState)") color magenta
                        [#
                    }
                }
            #]
        }
    }
    Goto engagedWaitContainer if [# CurrentState == "ENGAGED" #] else idle

   State engagedWaitContainer {
        [#
            if (WaitStartTime == 0L) {
                WaitStartTime = System.currentTimeMillis()
            }
            var Elapsed = System.currentTimeMillis() - WaitStartTime
            RemainingTime = 30000L - Elapsed
            if (RemainingTime <= 0L) RemainingTime = 1L
        #]
        println("$name | In attesa del container (Tempo residuo: ${RemainingTime}ms)...") color magenta
    }
    Transition t1
        whenTimeVar RemainingTime  -> handleTimeout
        whenRequest loadrequest    -> handleBusyRequest
        whenMsg containerSensed    -> handleContainer
        whenMsg outOfService       -> handleEngagedOutOfService

    State handleBusyRequest {
        onMsg(loadrequest : loadrequest(IOPORT_STATE)) {
            [#
                var CurrentHoldState = h.getHoldStateString()
                var SystemEngagedStr = "'system_engaged'"
            #]
            replyTo loadrequest with retrylater : retrylater($SystemEngagedStr, $CurrentHoldState)
            println("$name | retrylater($SystemEngagedStr, $CurrentHoldState)") color magenta
        }
    }
    Goto engagedWaitContainer

    State handleEngagedOutOfService {
        println("$name | GUASTO RILEVATO DURANTE ENGAGED!") color magenta
        [#
            IsOutOfService = true

            // Release dello slot che era stato prenotato
            if (CurrentReservedSlot != -1) {
                h.releaseSlot(CurrentReservedSlot)
                CurrentReservedSlot = -1
            }

            var CurrentHoldState = h.getHoldStateString()
            var StateStr = "'OUT_OF_SERVICE'"
            var MsgStr = "'Guasto sensore durante attesa'"
            var LedRed = "'red'"
            var ServiceDown = "'OUT_OF_SERVICE'"
        #]
        forward ioport -m led : led($LedRed)
        forward ioport -m updateWorkingState : updateWorkingState($ServiceDown)
        forward ioport -m updateHoldDisplay : updateHoldDisplay($StateStr, $CurrentHoldState, $MsgStr)
    }
    Goto outOfServiceState

    State handleTimeout {
        println("$name | TIMEOUT scaduto! Container non rilevato entro 30s.") color magenta
        [#
            if (CurrentReservedSlot != -1) {
                h.releaseSlot(CurrentReservedSlot)
                CurrentReservedSlot = -1
            }
            var CurrentHoldState = h.getHoldStateString()
            var StateStr = "'DISENGAGED'"
            var TimeoutMsgStr = "'Timer scaduto (30s)'"
        #]
        forward ioport -m updateHoldDisplay : updateHoldDisplay($StateStr, $CurrentHoldState, $TimeoutMsgStr)
    }
    Goto idle

    State handleContainer {
        println("$name | Container rilevato -> aggiornamento slot a OCCUPIED") color magenta
        [#
            if (CurrentReservedSlot != -1) {
                h.setSlotOccupied(CurrentReservedSlot)
                CurrentReservedSlot = -1
            }
            var CurrentHoldState = h.getHoldStateString()
            var StateStr = "'LOADED'"
            var LoadedMsgStr = "'Container preso in carico'"
        #]
        forward ioport -m updateHoldDisplay : updateHoldDisplay($StateStr, $CurrentHoldState, $LoadedMsgStr)
    }
    Goto idle

    State handleOutOfService {
        println("$name | ERRORE SENSORE: Ingresso in Out Of Service") color magenta
        [#
            IsOutOfService = true
            var CurrentHoldState = h.getHoldStateString()
            var StateStr = "'OUT_OF_SERVICE'"
            var MsgStr = "'Guasto sensore'"
            var LedRed = "'red'"
            var ServiceDown = "'OUT_OF_SERVICE'"
        #]
        forward ioport -m led : led($LedRed)
        forward ioport -m updateWorkingState : updateWorkingState($ServiceDown)
        forward ioport -m updateHoldDisplay : updateHoldDisplay($StateStr, $CurrentHoldState, $MsgStr)
    }
    Goto outOfServiceState

    State outOfServiceState {
        println("$name | Sistema BLOCCO: Out Of Service") color magenta
    }
    Transition t2
        whenRequest loadrequest -> handleRejectOutOfService

    State handleRejectOutOfService {
        onMsg(loadrequest : loadrequest(IOPORT_STATE)) {
            [#
                var CurrentHoldState = h.getHoldStateString()
                var CauseStr = "'out_of_service'"
            #]
            replyTo loadrequest with retrylater : retrylater($CauseStr, $CurrentHoldState)
        }
    }
    Goto outOfServiceState
}
```

L'attore ioport

L'attore funge da mediatore: reagisce agli stimoli provenienti dalla Web GUI tramite l'adapter, invia la richiesta a cargoservice, e riceve le notifiche di aggiornamento.

```
QActor ioport context ctxcargoservice {
   [#
        var SensorOccupied = false
        lateinit var Adapter: ioport.IOPortWsAdapter
    #]

    State s0 initial {
        println("$name | avvio attore IOPort") color blue
        [# Adapter = ioport.IOPortWsAdapter(myself, 7070) #]
    }
    Goto acceptEvents

    State acceptEvents {
        println("$name | in attesa di eventi...") color blue
    }
    Transition t0
        whenMsg button_pushed       -> handleButton
        whenMsg containerSensed     -> handleContainerMock
        whenMsg outOfService        -> handleOutOfServiceMock
        whenMsg updateHoldDisplay   -> handleCoreHoldUpdate
        whenMsg updateWorkingState  -> handleCoreWorkingState
        whenMsg led                 -> handleLedUpdate

        // Pressione pulsante GUI -> Invio Request a CargoService
    State handleButton {
        println("$name | Pressione pulsante ricevuta dalla GUI") color blue
        [# var IoportStateStr = if (SensorOccupied) "true" else "false" #]
        request cargoservice -m loadrequest : loadrequest($IoportStateStr)
    }
    Transition t1
        whenReply engaged    -> handleEngaged
        whenReply reject     -> handleReject
        whenReply retrylater -> handleRetryLater

    // Conversione Reply di CargoService -> Dispatch per la GUI
    State handleEngaged {
        onMsg(engaged : engaged(RESERVED_SLOT, HOLD_STATE)) {
            [#
                var ReservedSlot = payloadArg(0)
                var HoldState = payloadArg(1)
                var StateStr = "'ENGAGED'"
                var MsgStr = "'Slot " + ReservedSlot + " Riservato'"
                Adapter.updateHoldStatus(StateStr, HoldState, MsgStr)
            #]
            println("$name | Risposta engaged da parte di cargoservice: $StateStr, $HoldState, $MsgStr") color blue
        }
    }
    Goto acceptEvents

    State handleReject {
        onMsg(reject : reject(HOLD_STATE)) {
            [#
                var HoldState = payloadArg(0)
                var StateStr = "'REJECTED'"
                var MsgStr = "'stiva piena'"
                Adapter.updateHoldStatus(StateStr, HoldState, MsgStr)
            #]
            println("$name | Risposta reject da parte di cargoservice: $StateStr, $HoldState, $MsgStr") color blue
        }
    }
    Goto acceptEvents

    State handleRetryLater {
        onMsg(retrylater : retrylater(CAUSE, HOLD_STATE)) {
            [#
                var Cause = payloadArg(0)
                var HoldState = payloadArg(1)
                var StateStr = "'RETRY LATER'"
                var CauseStr = "'" + Cause + "'"
                Adapter.updateHoldStatus(StateStr, HoldState, CauseStr)
            #]
            println("$name | Risposta retrylater da parte di cargoservice: $StateStr, $HoldState, $CauseStr") color blue
        }
    }
    Goto acceptEvents

    // Forward verso Adapter WebSocket
    State handleCoreHoldUpdate {
        onMsg(updateHoldDisplay : updateHoldDisplay(STATE, HOLD_STATE, MSG)) {
            [# Adapter.updateHoldStatus(payloadArg(0), payloadArg(1), payloadArg(2)) #]
            println("$name | Richiesto aggiornamento del display") color blue
        }
    }
    Goto acceptEvents

   State handleCoreWorkingState {
        onMsg(updateWorkingState : updateWorkingState(STATE)) {
            [# Adapter.updateWorkingState(payloadArg(0)) #]
            println("$name | Richiesto aggiornamento del working state") color blue
        }
    }
    Goto acceptEvents

        // --- MOCK Led ---
    State handleLedUpdate {
        onMsg(led : led(STATE)) {
            [# Adapter.updateLed(payloadArg(0)) #]
            println("$name | Richiesto aggiornamento del led") color yellow
        }
    }
    Goto acceptEvents

   // --- MOCK Hardware ---
    State handleContainerMock {
        printCurrentMessage color red
        [#
                var SensorVal = currentMsg.msgContent()
                SensorOccupied = SensorVal.toBoolean()
                 #]
        println("$name | Variazione sensore container: $SensorOccupied") color green

        if [# SensorOccupied #] {
                forward cargoservice -m containerSensed : containerSensed(true)
        }
        }
        Goto acceptEvents

    State handleOutOfServiceMock {
        println("$name | Simulazione errore del sensore") color red
        forward cargoservice -m outOfService : outOfService(sensor_fault)
    }
    Goto acceptEvents
}
```

L'adattatore WebSocket: IOPortWsAdapter

Per colmare il divario tecnologico tra il mondo asincrono degli attori QAk e il browser web, l'adapter avvia un server Javalin su porta 7070, serve le risorse statiche della GUI e gestisce la connessione WebSocket full-duplex. La classe gestisce in modo thread-safe la coda dei client connessi e traduce i messaggi stringa ricevuti in IApplMessage formali tramite il metodo sendMsgToMyself:

```
package ioport;

import io.javalin.Javalin;
import io.javalin.http.staticfiles.Location;
import io.javalin.websocket.WsContext;
import it.unibo.kactor.ActorBasic;
import unibo.basicomm23.interfaces.IApplMessage;
import unibo.basicomm23.msg.ApplMessage;
import unibo.basicomm23.utils.CommUtils;

import java.time.Duration;
import java.util.concurrent.ConcurrentLinkedQueue;

public class IOPortWsAdapter {
    private final ActorBasic ownerActor;
    private final ConcurrentLinkedQueue connectedClients = new ConcurrentLinkedQueue<>();

    public IOPortWsAdapter(ActorBasic ownerActor, int port) {
        this.ownerActor = ownerActor;

        Javalin app = Javalin.create(config -> {
                config.staticFiles.add("utils/gui", Location.EXTERNAL);

            config.jetty.wsFactoryConfig(wsFactory -> {
                wsFactory.setIdleTimeout(Duration.ZERO);
            });
        }).start(port);

        app.ws("/ws/ioport", ws -> {
            ws.onConnect(ctx -> {
                connectedClients.add(ctx);
                System.out.println("GUI Client connesso via WebSocket!");
            });

            ws.onClose(ctx -> {
                connectedClients.remove(ctx);
                System.out.println("GUI Client disconnesso.");
            });

            ws.onMessage(ctx -> {
                String rawMsg = ctx.message();
                System.out.println("ricevuto messaggio: " + rawMsg);
                try {
                    // Parsing della stringa direttamente tramite la classe ApplMessage
                    IApplMessage msg = new ApplMessage(rawMsg);

                    // Inoltro all'attore ioport
                    ownerActor.sendMsgToMyself(msg);
                } catch (Exception e) {
                    System.err.println("Errore parsing IApplMessage: " + e.getMessage());
                }
            });
        });
    }

    public void updateHoldStatus(String state, String hold, String msg) {
        String cleanState = state != null ? state.replace("'", "") : "IDLE";
        String cleanHold  = hold != null ? hold.replace("'", "") : "[]";
        String cleanMsg   = msg != null ? msg.replace("'", "") : "";

        String content = String.format("hold_info('%s', '%s', '%s')", cleanState, cleanHold, cleanMsg);
        IApplMessage applMsg = CommUtils.buildDispatch("ioport", "hold_status", content, "webgui");
        broadcast(applMsg.toString());
    }

    public void updateWorkingState(String workingState) {
        String content = String.format("'%s'", workingState);
        IApplMessage applMsg = CommUtils.buildDispatch("ioport", "working_state", content, "webgui");
        broadcast(applMsg.toString());
    }
```

```
    }

    public void updateLed(String ledState) {
        String content = String.format("'%s'", ledState);
        IApplMessage applMsg = CommUtils.buildDispatch("ioport", "led", content, "webgui");
        broadcast(applMsg.toString());
    }

    public void broadcast(String message) {
        for (WsContext client : connectedClients) {
            if (client.session.isOpen()) {
                client.send(message);
            }
        }
    }
}
```

La Web GUI (ioport_gui.html e ioport_gui.js)

Il frontend web non contiene logica applicativa né di controllo:

- si limita a visualizzare lo stato del sistema (STATE), l'occupazione della stiva (HOLD) e i messaggi operativi.
- espone un pulsante per simulare la pressione fisica del pushbutton dell'IOPort.
- fornisce uno switch di testing per impostare lo stato di occupazione simulato del sensore (area antistante la porta).
- quando l'utente preme il pulsante, invia l'evento su WebSocket. L'eventuale countdown grafico a video ha uno scopo unicamente visivo per l'utente: la reale sorgente di verità del timeout risiede esclusivamente nel vincolo temporale governato da cargoservice.

Testing

Deployment

Il prototipo dello Sprint 1 è strutturato come un progetto autonomo ed eseguibile, pacchettizzato mediante Gradle.

Compilazione ed Esecuzione

Dalla radice del progetto dello Sprint 1 (`sprint1_project/`), eseguire da riga di comando:

```
./gradlew run
```

Il comando avvia il runtime QAk, istanzia il contesto locale con gli attori cargoservice e ioport, e solleva il server Javalin integrato nell'adapter.

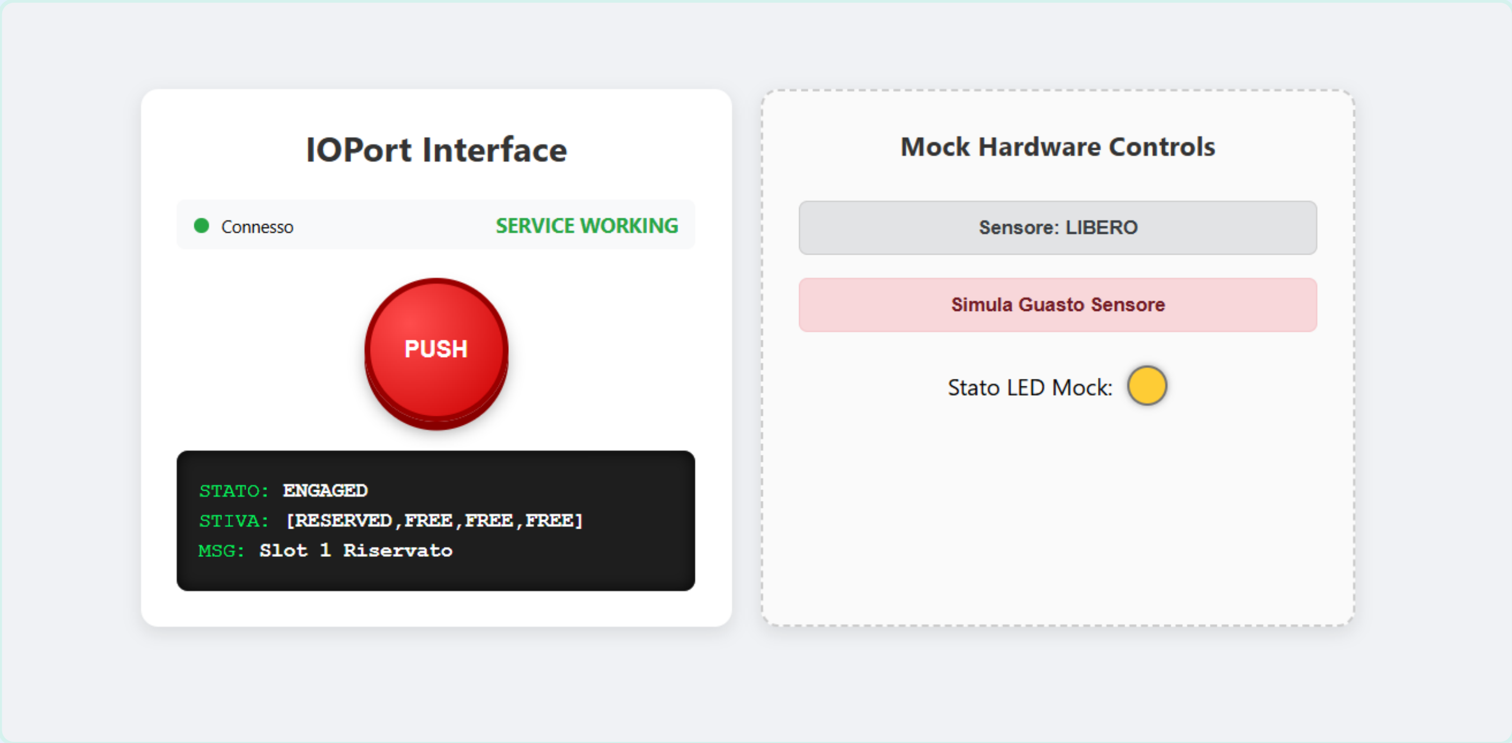
Mappatura delle porte di rete e servizi

Il sistema espone due endpoint di rete distinti con precise responsabilità funzionali:

Porta	Componente associato	Protocollo / Ruolo
8120	**`ctxcargoservice` (Runtime QAk)**	**TCP grezzo**: porta di ascolto infrastrutturale del contesto QAk. Nello Sprint 1 entrambi gli attori convivono nello stesso processo locale, ma la porta resta aperta a supporto delle future comunicazioni remote tra contesti distribuiti.
7070	**`IOPortWsAdapter` (Server Javalin)**	**HTTP / WebSocket**: serve le risorse statiche della WebGUI (HTML/CSS/JS) all'endpoint radice e gestisce il canale bidirezionale push/eventi sulla route /ws/ioport.

Guida all'uso per l'utente finale

Ad applicazione avviata, aprire il browser all'indirizzo `http://localhost:7070/ioport_gui.html`



```
STATO: ENGAGED
STIVA: [RESERVED,FREE,FREE,FREE]
MSG: Slot 1 Riservato
```

Per testare il funzionamento completo di cargoservice si è scelto di adottare un mock. L'implementazione del sensore e del led sarà inserita in uno Sprint successivo.

Definizione dello Sprint2

Requisiti selezionati per lo Sprint 2

- **Movimentazione del DDR Robot (cargorobot)**: A seguito dell'accettazione del carico e dell'avvenuto rilevamento del container nell'area sensore dell'IOPort, il robot deve muoversi dalla posizione HOME, raggiungere l'IOPort per prelevare il carico, trasferirlo allo slot5 per la marcatura, e infine depositarlo nello slot finale riservato (1–4) prima di rientrare in HOME.
- **Integrazione con RobotSmart26**: Interfacciare l'attore cargorobot con il microservizio esterno di navigazione (RobotSmart26), colmando l'abstraction gap del calcolo delle coordinate cartesiane e dei comandi di movimento su griglia.
- **Modellazione dell'attore marker (Slot 5)**: Introdurre il componente attivo che simula il dispositivo di etichettatura/scansione barcode nello slot 5, gestendo il tempo di marcatura e la notifica asincrona di completamento (`markingDone`) verso il robot.
- **Separazione dei contesti distribuiti**: Superare il vincolo del contesto unico (`ctxcargoservice`) riorganizzando il sistema su nodi computazionali distinti e comunicanti via protocollo QAk su TCP (`ctxcargoservice`, `ctxcargorobot`, `ctxioport`).

Goal dello Sprint 2

Progettare e realizzare l'attore `cargorobot` e integrarlo con l'ambiente virtuale `RobotSmart26` per eseguire la missione autonoma di trasferimento container (IOPort -> slot 5 -> slot assegnato -> HOME). Modellare il componente `marker` per la sincronizzazione del processo di marcatura nello slot 5 e distribuire l'architettura su contesti QAk eterogenei e separati.

Maintenance

Team di Sviluppo

💻 GitHub Project Repository

Arianna Buffoni
arianna.buffoni@studio.unibo.it

Desirée Pellegrini
desiree.pellegrini@studio.unibo.it

Gianluca Varisco
gianluca.varisco@studio.unibo.it